

DOKUMENTACJA TECHNICZNA PROJEKTU
QUILL
Post-kwantowy komunikator P2P z szyfrowaniem sesji
STUDIO PROJEKTOWE, SEMESTR 6

INFORMATYKA I SYSTEMY INTELIGENTNE

Autorzy projektu

Szymon Kowalski

Michał Król



EAIiB / Katedra Informatyki Stosowanej
Akademia Górniczo-Hutnicza im. Stanisława Staszica w
Krakowie
Kraków, Polska

Czerwiec 2026 wersja 1.0.0

Abstract

Quill to komunikator sieciowy napisany w C++23, w którym cała warstwa asymetryczna opiera się na standardach post-kwantowych NIST (FIPS 203 ML-KEM, FIPS 204 ML-DSA). Projekt implementuje własny protokół handshake, szyfrowanie sesji AES-256-GCM z ochroną przed replay, mechanizm TOFU, trwałe profile użytkownika, pokoje z hasłem oraz transfer plików z integralnością SHA-3. Aplikacja posiada interfejs ImGui oraz bibliotekę rdzeniową `quill_core` testowaną 105 testami Google Test i weryfikowaną w CI na GitHub Actions.

Słowa kluczowe: kryptografia post-kwantowa, ML-KEM, ML-DSA, AES-GCM, TOFU, komunikator P2P, C++23.

Contents

1	Wprowadzenie	6
1.1	Kontekst projektu	6
1.2	Cel i zakres	6
1.3	Struktura dokumentu	7
2	Motywacja i tło teoretyczne	7
2.1	Zagrożenie kwantowe	7
2.2	Algorytm Grovera a AES	7
2.3	Porównanie stosów kryptograficznych	8
2.4	Podstawy kryptografii na kratach	8
3	Model zagrożeń i założenia	9
3.1	Zagrożenia uwzględnione	9
3.2	Założenia zaufania	9
3.3	Co Quill <i>nie</i> chroni	10
4	Architektura systemu	10
4.1	Przegląd warstw	10
4.2	Model hub-and-spoke	10

4.3	Wielowątkowość	11
4.4	Pokoje (rooms) i izolacja wiadomości	11
4.5	Historia decyzji projektowych	12
4.6	Struktura katalogów projektu	12
5	Stos kryptograficzny	13
5.1	Biblioteki zewnętrzne	13
5.2	Poziomy bezpieczeństwa	13
5.3	KyberKEM — wymiana klucza	13
5.4	DilithiumSign — podpisy	14
5.5	Hkdf — wyprowadzanie klucza sesji	14
5.6	AesGcm — szyfrowanie sesji	14
5.7	Argon2 — ochrona kluczy na dysku	15
6	Protokół komunikacji	15
6.1	Format na linii (wire format)	15
6.2	Typy wiadomości	16
6.3	Handshake PQC — przebieg szczegółowy	16
6.3.1	Diagram sekwencji	16
6.3.2	SRV_HELLO (serwer → klient)	17
6.3.3	CLI_KEX (klient → serwer)	17
6.3.4	Inwarianty bezpieczeństwa handshake	17
6.4	Wiadomości CHAT	17
6.5	Perfect Forward Secrecy (PFS)	18
6.6	Transfer plików	18
6.6.1	Przebieg protokołu	18
6.6.2	Selective repeat	19
6.6.3	Relay serwera	19
7	Warstwa sieciowa	19

7.1	Klasa Socket	19
7.2	NetworkServer	19
7.3	NetworkClient	20
7.4	Ograniczenia sieciowe	20
8	Tożsamość, profile i zaufanie	20
8.1	IdentityManager	20
8.2	ProfileManager	21
8.3	TrustStore (TOFU)	21
9	Moduły implementacyjne — szczegóły	22
9.1	CryptoManager	22
9.2	MessageFormat	23
9.3	FileTransfer	23
9.4	ChatApp i ChatAppRender	23
9.5	CMake i cele build	23
10	Interfejs użytkownika	23
10.1	Technologia	23
10.2	Ekrany i panele	24
10.3	Handshake visualizer	24
10.4	Security Dashboard	24
10.5	Narzędzia audytowe	25
11	Scenariusze użycia	25
11.1	Scenariusz 1: Pierwsze uruchomienie (demo lokalne)	25
11.2	Scenariusz 2: Weryfikacja TOFU	25
11.3	Scenariusz 3: Transfer pliku	25
11.4	Scenariusz 4: Rotacja PFS	26
11.5	Scenariusz 5: Benchmark poziomów	26

12 Wyniki wydajnościowe (orientacyjne)	26
13 Testowanie	27
13.1 Strategia testów	27
13.2 Katalog testów (105 przypadków)	27
13.2.1 test_hkdf.cpp (6)	27
13.2.2 test_argon2.cpp (6)	27
13.2.3 test_aesgcm.cpp (13)	27
13.2.4 test_kyber.cpp (10)	28
13.2.5 test_dilithium.cpp (13)	28
13.2.6 test_identity.cpp (10)	28
13.2.7 test_profile.cpp (11)	28
13.2.8 test_roomstore.cpp (6)	28
13.2.9 test_truststore.cpp (8)	28
13.2.10 test_messageformat.cpp (3)	28
13.2.11 test_filetransfer.cpp (14)	29
13.2.12 test_cryptomanager.cpp (5)	29
13.3 Uruchamianie testów	29
13.4 CI GitHub Actions	29
14 Budowa i wdrożenie	30
14.1 Wymagania systemowe	30
14.2 Instalacja liboqs (Arch Linux)	30
14.3 Pełna budowa z GUI	30
14.4 Pakietowanie (opcjonalne)	31
15 Analiza bezpieczeństwa — podsumowanie	31
15.1 Mocne strony	31
15.2 Słabe strony (akceptowane)	31
15.3 Porównanie z TLS 1.3 + hybryda PQC	31

16	Znane ograniczenia i dług techniczny	32
17	Podsumowanie	32
17.1	Podział prac	32
17.2	Osiągnięcia	33
17.3	Możliwe kierunki rozwoju (poza zakresem studiów)	33
A	Załącznik A: Zmienne środowiskowe	34
B	Załącznik B: Rozmiary typowych artefaktów kryptograficznych	34
C	Załącznik C: Konwencje nazewnictwa peer_id w TOFU	34
D	Załącznik D: Przykład zaszyfrowanej wiadomości CHAT (logiczny)	34
E	Załącznik E: Format pliku rooms.json	35
F	Załącznik F: Format pliku known_hosts	35
G	Załącznik G: Lista plików źródłowych quill_core	36

1 Wprowadzenie

1.1 Kontekst projektu

Projekt **Quill** powstał w ramach przedmiotu *Studio Projektowe* na 6. semestrze studiów inżynierskich na Akademii Górniczo-Hutniczej w Krakowie, na kierunku Informatyka i Systemy Inteligentne. Celem było zaprojektowanie i implementacja działającego systemu komunikacji, który nie opiera się na klasycznej kryptografii asymetrycznej podatnej na ataki kwantowe, lecz na algorytmach zatwierdzonych przez NIST w 2024 roku.

Quill nie jest wrapperem TLS ani gotowym protokołem aplikacyjnym — implementuje własny stos od warstwy transportowej (TCP) po szyfrowanie wiadomości, z pełną kontrolą nad każdym krokiem handshake i każdym bajtem na linii. Taki poziom szczegółowości jest celowy: projekt ma być obronny akademicko w ramach Studio Projektowego i pokazywać pełny cykl projektowania bezpiecznego systemu komunikacji.

1.2 Cel i zakres

Cel główny: zbudować komunikator, w którym:

- a) wymiana klucza i uwierzytelnianie opierają się wyłącznie na algorytmach post-kwantowych,
- b) sesja jest szyfrowana AES-256-GCM z wiązaniem kontekstu (AAD) i ochroną przed replay,
- c) użytkownik ma trwałą tożsamość kryptograficzną chronioną hasłem na dysku,
- d) zaufanie do peera jest zarządzane mechanizmem TOFU (jak w SSH),
- e) możliwy jest bezpieczny transfer plików z weryfikacją integralności.

Zakres zrealizowany obejmuje m.in.: handshake PQC, trzy poziomy bezpieczeństwa (FAST/BALANCED/MAX), Perfect Forward Secrecy, pokoje rozmów z trwałym zapisem i hasłem, interfejs graficzny z narzędziami audytowymi, 105 testów automatycznych oraz pipeline CI.

Poza zakresem (świadomie odłożone): NAT traversal (UDP hole punching), prawdziwy end-to-end bez zaufania do serwera-relay, wersjonowanie schematu JSON protokołu, streaming plików bez buforowania w RAM.

1.3 Struktura dokumentu

Dokument opisuje kolejno: tło teoretyczne i model zagrożeń (rozdz. 2–3), architekturę systemu (rozdz. 4), stos kryptograficzny (rozdz. 5), szczegółowy protokół komunikacji (rozdz. 6), warstwę sieciową (rozdz. 7), tożsamość i zaufanie (rozdz. 8), moduły implementacyjne (rozdz. 9), interfejs użytkownika (rozdz. 10), scenariusze użycia (rozdz. 11), testowanie (rozdz. 12), budowę i wdrożenie (rozdz. 13), analizę bezpieczeństwa (rozdz. 14), ograniczenia (rozdz. 15), podsumowanie (rozdz. 16) oraz załączniki.

2 Motywacja i tło teoretyczne

2.1 Zagrożenie kwantowe

Klasyczna kryptografia asymetryczna (RSA, DSA, ECDH, ECDSA) opiera się na problemach, które na komputerze kwantowym rozwiązuje algorytm Shora w czasie wielomianowym. Oznacza to, że długoterminowo przechwycone dzisiejsze sesje zaszyfrowane RSA/ECDH mogą zostać odszyfrowane po pojawieniu się wystarczająco dużego komputera kwantowego (*harvest now, decrypt later*).

W 2024 roku NIST opublikował pierwsze standardy kryptografii post-quantowej:

- **FIPS 203** — ML-KEM (Module-Lattice Key Encapsulation Mechanism), wcześniej znany jako CRYSTALS-Kyber,
- **FIPS 204** — ML-DSA (Module-Lattice Digital Signature Algorithm), wcześniej CRYSTALS-Dilithium,
- **FIPS 205** — SLH-DSA (SPHINCS+) — podpisy hash-based (w Quill nieużywane bezpośrednio, ale dostępne w liboqs).

Quill implementuje ML-KEM i ML-DSA (oraz Falcon-512 w poziomie FAST) jako zamienniki całej klasycznej asymetrii.

2.2 Algorytm Grovera a AES

Symetryczne szyfrowanie AES-256 nie jest podatne na Shora w taki sam sposób jak RSA. Algorytm Grovera redukuje efektywną siłę klucza o połowę — AES-256 daje ~ 128 bitów bezpieczeństwa kwantowego, co nadal jest uznawane za bezpieczne.

Dlatego Quill zachowuje AES-256-GCM jako szyfr sesji, koncentrując wysiłek PQC na wymianie klucza i podpisach.

2.3 Porównanie stosów kryptograficznych

Funkcja	Klasycznie	Quill (PQC)
Wymiana klucza	ECDH, RSA-OAEP	ML-KEM (Kyber-512/768/1024)
Podpis / auth	ECDSA, RSA-PSS	ML-DSA-65/87, Falcon-512
Szyfrowanie sesji	AES-256-GCM	AES-256-GCM (bez zmian)
KDF	często HKDF / TLS PRF	HKDF-SHA256 (RFC 5869)
Hash integralności	SHA-256	SHA-3-256

Table 1: Mapowanie funkcji kryptograficznych w Quill.

2.4 Podstawy kryptografii na kratkach

Kyber (ML-KEM) i Dilithium (ML-DSA) opierają się na trudności problemów na strukturalnych kratkach (Module-LWE, Module-SIS). Intuicyjnie: klucz publiczny to „zaszumione” równania liniowe, a klucz prywatny pozwala odsumować wynik. Bezpieczeństwo opiera się na założeniu, że najlepsze znane ataki wymagają wykładniczego czasu na klasycznym i kwantowym komputerze (w odróżnieniu od faktoryzacji i logarytmu dyskretnego).

Quill nie implementuje tych algorytmów od zera — używa biblioteki **liboqs** (Open Quantum Safe), która dostarcza referencyjne implementacje zweryfikowane w konkursie NIST. Własna implementacja obejmuje natomiast protokół, KDF, TOFU, serializację i logikę aplikacji.

3 Model zagrożeń i założenia

3.1 Zagrożenia uwzględnione

Zagrożenie	Mechanizm obrony w Quill
Podśluch na łączu	AES-256-GCM na każdej wiadomości; klucz sesji z ML-KEM
Atak MITM przy handshake	Podpisy ML-DSA/Falcon + TOFU fail-closed
Replay wiadomości czatu	Monotoniczny seq wiązany przez GCM AAD
Manipulacja cipher-textu	Tag GCM 128-bit; odrzucenie przy weryfikacji
Zmiana klucza peera	TOFU MISMATCH — blokada przed KEM encaps
Kradzież pliku klucza z dysku	Argon2id + AES-256-GCM (format QID2)
Uszkodzenie danych w transferze pliku	SHA-3 per chunk + hash całości w FILE_END
Wstrzyknięcie chunka z innego transferu	AAD FILE tid index + chunk_hash

Table 2: Zagrożenia i odpowiadające im mechanizmy.

3.2 Założenia zaufania

- Użytkownik chroni passphrase profilu — to jedyna obrona offline przy kradzieży pliku klucza.
- Porównanie fingerprintów out-of-band (telefon, QR) jest odpowiedzialnością użytkownika — TOFU sam nie gwarantuje weryfikacji tożsamości przy pierwszym kontakcie.
- **Serwer jest zaufanym relayem** — widzi plaintext wiadomości. Każdy hop klient↔serwer jest szyfrowany, ale serwer odszyfrowuje i ponownie szyfruje dla odbiorców.
- Sieć TCP jest niezawodna w dostarczaniu bajtów w kolejności (brak UDP, brak NAT traversal).

3.3 Co Quill *nie* chroni

- Atak na działający proces (keyloggery, pamięć RAM po odblokowaniu profilu).
- Kompromitacja serwera-hosta (serwer widzi treść wiadomości).
- Denial of service (brak rate limiting, brak ochrony przed floodem TCP).
- Ukrywanie metadanych (adresy IP, czasy połączeń, rozmiary pakietów).

4 Architektura systemu

4.1 Przegląd warstw

System podzielony jest na warstwę prezentacji (ImGui), logikę aplikacji (ChatApp) oraz bibliotekę rdzeniową quill_core, która nie zależy od GUI.

Listing 1: Hierarchia modułów.

```
main.cpp

    ChatApp (frontend/)
        ProfileManager, TrustStore (UI + aktywacja)
        NetworkServer / NetworkClient (w tki sieciowe)
        FileReceiver, wizualizacja handshake

    quill_core (biblioteka statyczna)
        crypto/      CryptoManager, KyberKEM,
        DilithiumSign,
                    Hkdf, AesGcm, Argon2,
        IdentityManager
        network/     Socket, framing TCP
        protocol/    MessageFormat, FileTransfer
```

Decyzja projektowa: separacja quill_core od UI umożliwia uruchomienie 105 testów i CI bez GLFW/OpenGL/ImGui. Ta sama logika kryptograficzna działa w aplikacji i w testach integracyjnych (socketpair).

4.2 Model hub-and-spoke

Quill nie jest czystym P2P mesh — działa w modelu *gwiazdy*:

- jedna instancja uruchamia **serwer** (nasłuch TCP),
- pozostałe instancje łączą się jako **klienci**,
- serwer przekazuje wiadomości między użytkownikami w pokojach (rooms),
- każdy klient ma osobny klucz sesji AES z serwerem (osobny handshake).

Zalety: prosty broadcast, jeden punkt spotkania, łatwe demo na jednej maszynie (localhost). Wada: serwer widzi plaintext — opisana w rozdz. 15.

4.3 Wielowątkowość

Serwer obsługuje wielu klientów — każde połączenie TCP ma dedykowany wątek `std::thread (server_client_handler)`. Główny wątek renderuje ImGui (60 FPS). Współdzielony stan (lista klientów, pokoje) chroniony jest mutexem `m_clients_mtx`. Licznik wiadomości `m_msg_count` jest atomowy.

4.4 Pokoje (rooms) i izolacja wiadomości

Serwer utrzymuje mapę `m_rooms`: nazwa pokoju → zbiór klientów. Domyślny pokój to `general`. Klient wybiera pokój w UI; serwer przypisuje go w strukturze `ConnectedClient::room`.

Broadcast (`broadcast_to_room`) wysyła wiadomość do wszystkich uczestników pokoju oprócz nadawcy. Wiadomości z innych pokoi nie docierają do odbiorców — izolacja na poziomie aplikacji (serwer filtruje po polu `room`).

Każdy klient w pokoju ma osobny klucz sesji AES (osobny handshake przy połączeniu). Serwer musi więc odszyfrować wiadomość od nadawcy i zaszyfrować ją osobno dla każdego odbiorcy — stąd model hub-and-spoke opisany w rozdz. 4.

RoomStore (`rooms.json` w katalogu profilu) utrwała listę pokoi między uruchomieniami serwera. Pokój `general` jest domyślny i nieusuwalny. Pokoje chronione hasłem przechowują weryfikator Argon2id (salt + hash). Protokół obsługuje m.in. `CREATE_ROOM`, `ROOM_CREATED`, `ROOM_EXISTS`, `ROOM_DELETED`, `JOIN_DENIED` oraz `ROOM_LIST` z flagą `protected`. Poziom PQC ustala serwer (`SERVER_CONFIG` przed handshake).

4.5 Historia decyzji projektowych

Decyzja	Alternatywa	Uzasadnienie
Własny protokół liboqs zamiast własnego KEM	TLS 1.3 + PQC hybrid Implementacja Kyber od zera	Pełna kontrola, cel edukacyjny Bezpieczeństwo, czas
TOFU zamiast PKI JSON na TCP	Certyfikaty X.509 Protobuf / binarny	Zakres semestralny, jak SSH Czytelność, debug, Packet In- spector
ImGui zamiast Qt quill_core bez UI	Qt Widgets / web UI Monolit	Lekkość, szybki prototyp Testowalność, CI headless
Hub relay	Mesh P2P	Prostszy broadcast, demo 1- serwer

Table 3: Wybrane decyzje architektoniczne.

4.6 Struktura katalogów projektu

Listing 2: Drzewo repozytorium (skrót).

```

Quill/
  main.cpp                # entry point GLFW + ImGui
  CMakeLists.txt
  src/
    crypto/               # 10 modu w kryptograficznych
    network/              # Socket, Server, Client
    protocol/             # MessageFormat, FileTransfer
    frontend/             # ChatApp, Theme, render
  tests/                  # 12 plik w testowych, 105 przypadk w
  third_party/
    imgui/                # git submodule v1.92.8
    portable-file-dialogs/ # wyb r plik w w GUI
  docs/                   # dokumentacja (ten plik)
  .github/workflows/ci.yml

```

5 Stos kryptograficzny

5.1 Biblioteki zewnętrzne

Biblioteka	Licencja	Rola w Quill
liboqs	MIT	ML-KEM, ML-DSA, Falcon
OpenSSL	Apache 2.0	AES-GCM, HKDF, SHA-3, RAND
libargon2	CC0/Apache	Fallback Argon2id (OpenSSL < 3.2)
nlohmann/json	MIT	Serializacja wiadomości
Dear ImGui	MIT	Interfejs graficzny
GLFW	+ zlib / —	Okno i kontekst renderowania
OpenGL		
Google Test	BSD-3	Framework testów

5.2 Poziomy bezpieczeństwa

Enum `SecurityLevel` definiuje trzy tryby używane przy połączeniu:

Poziom	KEM	Podpis	NIST	Zastosowanie
FAST	Kyber-512	Falcon-512	Level 1	IoT, niskie opóźnienie
BALANCED	Kyber-768	ML-DSA-65	Level 3	Domyślny, kompromis
MAX	Kyber-1024	ML-DSA-87	Level 5	Dane krytyczne

Table 4: Mapowanie poziomów na algorytmy liboqs.

Poziom wybierany jest w UI przed połączeniem i musi być zgodny po obu stronach (negocjacja w handshake). Benchmarki w aplikacji mierzą czasy `keygen/encaps/decaps/sign/verify` dla każdego poziomu.

5.3 KyberKEM — wymiana klucza

Klasa `KyberKEM` opakowuje liboqs:

- `generate_keypair()` — generacja efemerycznej pary na handshake (PFS),
- `encapsulate(pub) → (ciphertext, shared_secret)`,
- `decapsulate(ct, sk) → shared_secret`.

Nowa para Kyber przy każdym handshake oznacza, że kompromitacja klucza sesji nie ujawnia przyszłych sesji (*forward secrecy* na poziomie KEM). Klucz DSA pozostaje trwały — służy tożsamości, nie sekretowi sesji.

5.4 DilithiumSign — podpisy

Klasa `DilithiumSign` obsługuje ML-DSA-65/87 oraz Falcon-512 (poziom FAST):

- `sign(message, secret_key) → podpis`,
- `verify(message, signature, public_key) → bool`,
- weryfikacja **zawsze** przed użyciem materiału KEM (inwariant w `CryptoManager`).

Podpisy wiążą: (1) klucz publiczny Kyber serwera w `SRV_HELLO`, (2) ciphertext KEM klienta w `CLI_KEX`.

5.5 Hkdf — wyprowadzanie klucza sesji

Surowy `shared_secret` z ML-KEM nie jest używany bezpośrednio jako klucz AES. HKDF-SHA256 (RFC 5869, OpenSSL `EVP_KDF`) wyprowadza 32-bajtowy klucz:

- **IKM**: `shared_secret` z Kyber,
- **salt**: `transkrypt handshake = kyber_pub || kem_ciphertext`,
- **info**: `quill-aes256gcm-session-v1|<LEVEL>` — separacja domen,
- po HKDF: `OPENSSL_cleanse` na `shared_secret`.

Wiązanie z transkryptem gwarantuje, że klucz sesji jest unikalny dla *tej konkretnej* wymiany, nawet gdyby ten sam `shared_secret` powtórzył się (teoretycznie nieprawdopodobne, ale wymóg defensywny).

5.6 AesGcm — szyfrowanie sesji

AES-256-GCM przez OpenSSL `EVP`:

- nonce 12 bajtów losowych (`RAND_bytes`) na każde szyfrowanie,

- tag uwierzytelniający 16 bajtów,
- opcjonalny AAD (Additional Authenticated Data) — używany dla CHAT i FILE_CHUNK,
- odrzucenie przy złym kluczu, zmanipulowanym ciphertext/tag lub niespójnym AAD.

5.7 Argon2 — ochrona kluczy na dysku

Format **QID2** szyfruje klucz prywatny DSA:

- Argon2id: $t = 3$ iteracje, $m = 64$ MiB pamięci (RFC 9106),
- losowa sól 16 B, parametry zapisane w pliku,
- wyprowadzony klucz 32 B $\rightarrow$ AES-256-GCM,
- złe hasło = błąd weryfikacji tagu GCM (jeden komunikat dla złego hasła i manipulacji).

Na systemach z OpenSSL ≥ 3.2 używany jest EVP_KDF ARGON2ID; na starszych (Ubuntu w CI) — biblioteka `libargon2`.

6 Protokół komunikacji

6.1 Format na linii (wire format)

Każda wiadomość na TCP ma prefiks długości:

1. 4 bajty `uint32_t` — długość payloadu (little-endian),
2. N bajtów — payload (JSON tekstowy lub binarny zakodowany w JSON).

Funkcje `Socket::send_bytes` / `receive_bytes` realizują semantykę *send all* / *recv all* — gwarantują dostarczenie kompletnej ramki.

6.2 Typy wiadomości

Enum `MsgType` w `MessageFormat.h`:

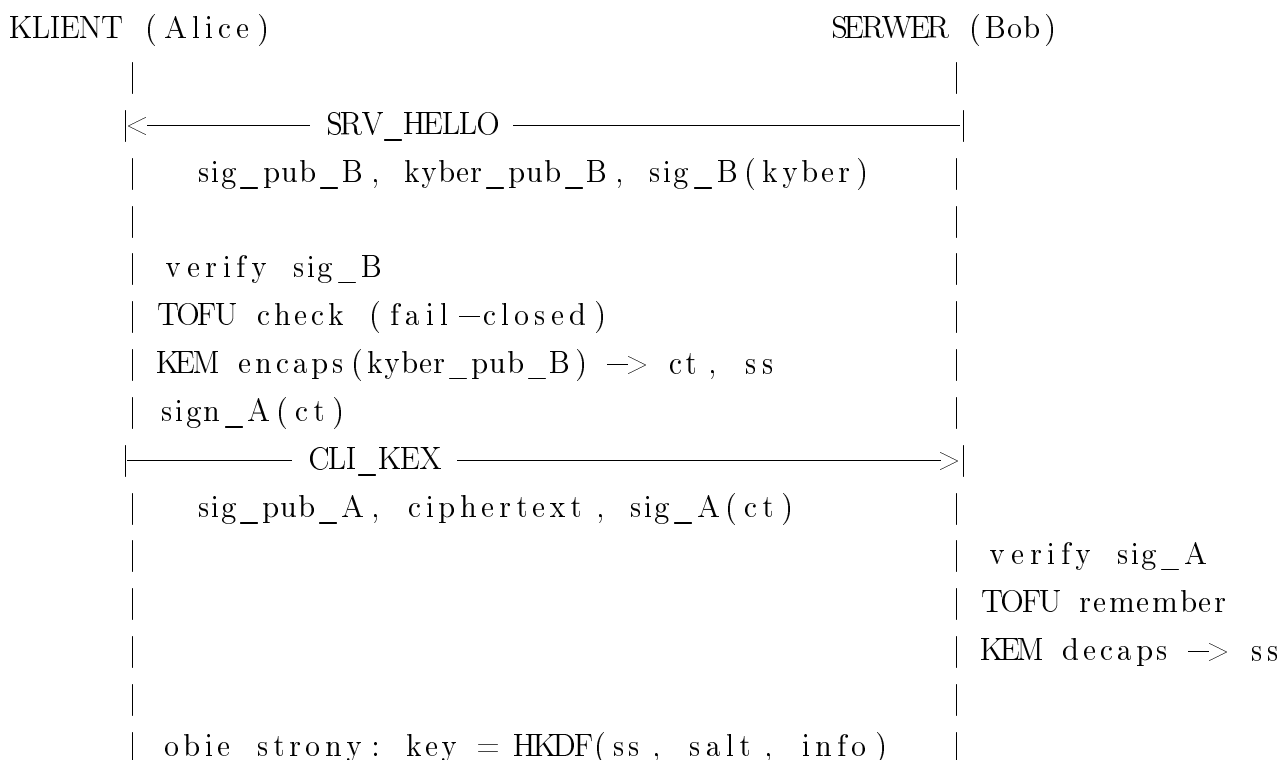
Typ	Faza	Opis
HANDSHAKE	Negocjacja	SRV_HELLO, CLI_KEX (JSON bez szyfrowania)
CHAT	Sesja	Zaszyfrowana wiadomość tekstowa
FILE_START	Transfer	Metadane pliku
FILE_CHUNK	Transfer	Zaszyfrowany fragment 64 KB
FILE_END	Transfer	Hash SHA-3-256 całego pliku
FILE_NACK	Transfer	Lista brakujących <code>chunk_index</code>
SYSTEM_CMD	Sterowanie	Polecenia wewnętrzne (PFS, pokoje)

Struktura `SecureMessage` serializuje pola do JSON. Dane binarne (nonce, payload, hash) kodowane są jako tablice bajtów — *nie* jako `json::binary`, które nie przeżywa `dump()`/`parse()` (bug naprawiony i pokryty testami).

6.3 Handshake PQC — przebieg szczegółowy

6.3.1 Diagram sekwencji

Listing 3: Handshake Quill (uproszczony).



===== sesja AES-256-GCM =====

6.3.2 SRV_HELLO (serwer → klient)

Przykładowa struktura JSON (pola binarne skrócone):

```
{
  "type": "SRV_HELLO",
  "sig_pub":    [/* bajty klucza publicznego DSA */],
  "kyber_pub":  [/* bajty klucza publicznego Kyber */],
  "sig":        [/* podpis kyber_pub kluczem DSA serwera */]
}
```

6.3.3 CLI_KEX (klient → serwer)

```
{
  "type": "CLI_KEX",
  "sig_pub":    [/* klucz publiczny DSA klienta */],
  "ciphertext": [/* wynik KEM encaps */],
  "sig":        [/* podpis ciphertext kluczem DSA klienta */]
}
```

6.3.4 Inwarianty bezpieczeństwa handshake

1. `verify()` podpisu **przed** `encaps()` / `decaps()`.
2. TOFU MISMATCH → wyjątek `TofuMismatchError` **przed** wysłaniem ciphertextu KEM.
3. Klucz sesji wyłącznie przez HKDF; `shared_secret` czyszczony z pamięci.
4. Kyber keypair efemeryczny; DSA z `IdentityManager::load_or_generate`.

6.4 Wiadomości CHAT

Po handshake każda wiadomość czatu:

1. Nadawca inkrementuje `send_seq`, buduje `AAD = "CHAT|" + std::to_string(seq)`.
2. Plaintext szyfrowany AES-256-GCM z losowym nonce 12 B.

3. JSON zawiera: `type`, `sender`, `seq`, `nonce`, `payload` (ciphertext+tag), `timestamp`.
4. Odbiorca deszyfruje, weryfikuje AAD, odrzuca jeśli `seq <= recv_seq`.

Ochrona przed replay jest *fail-closed*: podmiana numeru sekwencji w JSON bez ponownego szyfrowania psuje tag GCM (AAD musi się zgadzać). Ponowne wysłanie starej wiadomości z tym samym seq jest odrzucane logicznie.

Reset liczników: przy każdym nowym handshake i rotacji PFS liczniki `send_seq`/`recv_seq` zerowane — stary szyfrogram i tak nie deszyfruje się nowym kluczem.

6.5 Perfect Forward Secrecy (PFS)

Rotacja kluczy inicjowana jest:

- ręcznie z UI (przycisk Rotate Keys),
- automatycznie na serwerze co N wiadomości (konfigurowalne).

Przebieg: nowy handshake PQC na istniejącym połączeniu TCP (bez disconnect). Serwer ustawia flagę `pending_pfs_rotation`; wątek handlera klienta wykonuje `perform_pfs_rotation`. `Socket::try_receive_bytes(500ms)` pozwala obsłużyć rotację bez blokowania na brak wiadomości od klienta.

6.6 Transfer plików

6.6.1 Przebieg protokołu

1. **FILE_START** — `transfer_id` (UUID), `file_name`, `file_size`, `total_chunks`.
2. **FILE_CHUNK** $\times N$ — każdy chunk max 64 KB plaintextu:
 - szyfrowanie AES-256-GCM z unikalnym nonce,
 - `AAD = "FILE|" + transfer_id + "|" + chunk_index`,
 - `chunk_hash = SHA3-256(plaintext chunka)` w JSON — weryfikacja *przed* buforowaniem,
3. **FILE_END** — `file_hash = SHA3-256(cały plik)`.
4. **FILE_NACK** (opcjonalnie) — `missing: [indeksy]`; nadawca retransmituje z **nowym nonce** (max 5 rund).

6.6.2 Selective repeat

Klasa `FileSenderSession` trzyma plaintext wszystkich chunków w pamięci do momentu zakończenia transferu (umożliwia retransmisję). `FileReceiver` buforuje chunki w mapie `chunk_index` $\rightarrow$ `bytes` i scala przez `assemble()` po kompletności.

Jeśli `FILE_END` dotrze przed wszystkimi chunkami, odbiorca ustawia `awaiting_end` = `true` i czeka na uzupełnienie przez NACK. Po przekroczeniu 5 rund NACK transfer kończy się błędem.

6.6.3 Relay serwera

Serwer odszyfrowuje chunk od nadawcy i szyfruje ponownie dla każdego odbiorcy w pokoju tym samym kluczem sesji co chat, zachowując ten sam AAD i `chunk_hash`.

7 Warstwa sieciowa

7.1 Klasa Socket

Abstrakcja nad deskryptorem POSIX:

- `send_bytes / receive_bytes` — blokujące I/O z framingiem,
- `try_receive_bytes(timeout_ms)` — nieblokujące z timeoutem (PFS, cleanup),
- RAII — zamknięcie FD w destruktorze.

7.2 NetworkServer

- `bind(host, port) + listen()`,
- pętla `accept()` w wątku głównym serwera,
- każdy klient: nowy `Socket`, handshake przez `CryptoManager::server_handshake`, rejestracja w `m_clients`,
- `broadcast_to_room(room, packet)` — wysyłka do wszystkich w pokoju oprócz nadawcy,
- cleanup martwych połączeń: usunięcie z `m_clients` i `m_rooms`.

7.3 NetworkClient

- `connect(host, port)`,
- handshake przez `CryptoManager::client_handshake` z `peer_id = "srv:" + host + ":" + port`,
- pętla odbioru w wątku roboczym, kolejka wiadomości do UI.

7.4 Ograniczenia sieciowe

Quill używa wyłącznie TCP IPv4/IPv6. Brak:

- UDP hole punching,
- serwera rendezvous,
- TLS wrappera (niepotrzebny — własny PQC handshake),
- kompresji payloadu.

Oba peeri muszą być osiągalne pod znanym adresem IP i portem (np. localhost dla demo lub sieć LAN w laboratorium).

8 Tożsamość, profile i zaufanie

8.1 IdentityManager

Trwała tożsamość kryptograficzna — para kluczy DSA generowana raz i przechowywana na dysku.

Lokalizacja: `~/.quill/profiles/<nazwa>/` (aktywowana przez profil) lub fallback `$QUILL_IDENTITY_DIR / ~/.quill/identity`.

Bezpieczeństwo plików:

- katalog: `chmod 0700`,
- plik klucza: `chmod 0600`,
- zapis atomowy: plik tymczasowy + `rename()`,
- odmowa wczytania przy zbyt szerokich uprawnieniach (wzorzec OpenSSH),

- self-test sign/verify po wczytaniu — uszkodzony klucz = wyjątek, nie cicha regeneracja,
- suma SHA-3-256 payloadu w pliku QID1 — detekcja korupcji bajtów.

Formaty:

- **QID1** — klucz plaintext (legacy, ostrzeżenie w UI),
- **QID2** — Argon2id + AES-256-GCM; automatyczna migracja QID1→QID2 przy odblokowaniu hasłem.

Fingerprint: SHA-3-256 klucza publicznego, pierwsze 12 bajtów w formacie XXXX-XXXX-XXXX-XXXX-XXXX-XXXX (np. A3F2-9C1B-44DE-F021-8B3C-7A09).

8.2 ProfileManager

Profil = katalog z `profile.json`, podkatalogiem `identity/`, plikami `known_hosts`, `rooms.json` oraz opcjonalnie `.session.lock` (blokada jednej aktywnej sesji GUI).

Tworzenie profilu:

- nazwa: whitelist znaków (ochrona przed path traversal),
- passphrase: min. 8 znaków, odrzucenie aaaaaaaa itp.,
- nowe profile zawsze `encrypted=true` (QID2).

Zmienne środowiskowe:

- `QUILL_PROFILES_DIR` — nadpisanie `~/.quill/profiles`,
- `QUILL_IDENTITY_DIR` — katalog tożsamości (fallback).

Logout: rozłącza sesję sieciową, czyści passphrase i klucze tajne z pamięci (`OPENSSL_cleanse`).
Usuwanie profilu wymaga potwierdzenia hasłem w oknie modalnym.

8.3 TrustStore (TOFU)

Plik `known_hosts` w katalogu profilu (JSON, `chmod 0600`).

Identyfikacja peerów:

- klient → serwer: "srv:<host>:<port>",
- serwer → klient: "cli:<fingerprint>" (klient nie ma stałego adresu).

Stany:

UNVERIFIED

Pierwszy kontakt; klucz zapisany; ostrzeżenie w UI.

KNOWN

Klucz zgodny z zapisem; fingerprint nie potwierdzony out-of-band.

VERIFIED

Użytkownik kliknął „Mark as Verified” po porównaniu fingerprintu.

MISMATCH

Inny klucz niż zapisany — połączenie zablokowane; wpis *nie* nadpisywany.

Usunięcie wpisu (`TrustStore::remove`) — jedyna droga po MISMATCH do ponownego połączenia ze zmienionym kluczem (świadoma decyzja użytkownika).

Porównanie kluczy odbywa się na pełnym SHA-3-256, nie na skróconym fingerprintcie wyświetlanym w UI.

9 Moduły implementacyjne — szczegóły

9.1 CryptoManager

Centralny moduł handshake. Metody publiczne:

- `client_handshake(sock, peer_id, on_step)`,
- `server_handshake(sock, on_step)`.

Zwraca `HandshakeResult`: `session_key` (32 B), `peer_fingerprint`, `peer_trust`, `total_ms`.

`ChatApp::do_client_handshake` i `do_server_handshake` to cienkie wrappery dodające logi i wizualizator kroków — cała logika bezpieczeństwa jest w `quill_core`.

9.2 MessageFormat

Serializacja/deserializacja `SecureMessage`. Kluczowa poprawka: pola binarne jako tablice JSON `[0,1,2,...]`, nie `json::binary_t`, które ginęło przy tekstowym `dump()`.

9.3 FileTransfer

FileSender — statyczne metody wysyłki i `chunk_aad()`.

FileSenderSession — stan sesji z buforem chunków do retransmisji.

FileReceiver — mapa aktywnych transferów, logika NACK, `try_finalize()`.

Stałe: `FILE_CHUNK_SIZE = 65536`, `FILE_MAX_NACK_ROUNDS = 5`.

9.4 ChatApp i ChatAppRender

ChatApp — logika: profile, sieć, szyfrowanie wiadomości, pokoje, PFS, transfer plików, benchmarki.

ChatAppRender.cpp — renderowanie ImGui (oddzielenie od logiki).

Theme.h — autorski motyw kolorystyczny (ciemny, akcent fioletowy).

9.5 CMake i cele build

Cel	Typ	Opis
<code>quill_core</code>	STATIC	Krypto + sieć + protokół
<code>imgui</code>	STATIC	Dear ImGui + backends (tylko GUI)
<code>Quill</code>	EXECUTABLE	Aplikacja (gdy <code>QUILL_BUILD_GUI=ON</code>)
<code>quill_tests</code>	EXECUTABLE	Google Test (jeśli znaleziono GTest)

Opcja `QUILL_BUILD_GUI=OFF` pomija GLFW, OpenGL i ImGui — używana w CI.

10 Interfejs użytkownika

10.1 Technologia

Dear ImGui v1.92.8 (git submodule), backend GLFW + OpenGL 3.3. Renderowanie w pętli głównej `main.cpp`; dane sieciowe w wątkach roboczych, ko-

munikacja z UI przez kolejki i mutexy.

10.2 Ekrany i panele

Panel	Funkcja
Login	Lista profili, tworzenie (passphrase $\times 2$), odblokowanie, usuwanie profilu
Setup	Tryb Server/Client, host, port; poziom PQC tylko po stronie serwera
Chat	Wiadomości z zawijaniem, wybór pokoju, tworzenie/usuwanie pokoi
Handshake	Wizualizator kroków z czasami ms, stan TOFU
Security Dashboard	Szacunki czasu złamania, rozmiary kluczy
Benchmarks	Pomiar FAST/BALANCED/MAX (keygen, encaps, sign...)
Tools	Packet Inspector (hex dump), symulator Tamper (MITM)
File transfer	Wybór pliku (portable-file-dialogs), panel postępu (scroll), folder pobrań
Trusted Peers	Lista known_hosts, Verify, Remove
Profile menu	Fingerprint, logout, delete profile (modal + hasło)

10.3 Handshake visualizer

Każdy krok handshake wywołuje callback `StepCallback(label, time_ms)`. UI zapisuje listę `HandshakeStep` i rysuje pasek postępu z czasami, np.:

- „Server Signature Verified (ML-DSA)” — 1.2 ms,
- „KEM Encapsulation (ML-KEM)” — 0.4 ms,
- „Session Key Derived (HKDF-SHA256)” — 0.1 ms.

10.4 Security Dashboard

Wyświetla szacunki (`SecurityEstimate`):

- czas złamania na superkomputerze klasycznym,
- czas złamania na komputerze kwantowym,
- werdykt kolorystyczny (bezpieczny / podatny).

Dla RSA/ECDH werdykt byłby „podatny”; dla ML-KEM/ML-DSA — „odporny na kwantowe”.

10.5 Narzędzia audytowe

Packet Inspector — podgląd ostatnich pakietów w formie hex + ASCII (edukacyjny, do audytu protokołu).

Tamper simulator — celowa modyfikacja bajtu w pakiecie handshake lub sesji, demonstracja odrzucenia przez weryfikację GCM/podpisu.

11 Scenariusze użycia

11.1 Scenariusz 1: Pierwsze uruchomienie (demo lokalne)

1. Uruchom `./Quill` — ekran logowania.
2. Utwórz profil **Alice** z hasłem (min. 8 znaków).
3. Zaloguj się; zanotuj fingerprint w pasku statusu.
4. Wybierz tryb **Server**, port 9999, poziom **BALANCED**; Start.
5. W drugiej instancji: profil **Bob**, tryb **Client**, localhost:9999.
6. Handshake automatyczny; TOFU: Bob widzi **UNVERIFIED** dla serwera.
7. Wymiana wiadomości w pokoju **general**.

11.2 Scenariusz 2: Weryfikacja TOFU

1. Po pierwszym połączeniu porównaj fingerprint serwera out-of-band (np. obie instancje na jednym ekranie).
2. W panelu **PEER TRUST** kliknij „Mark as Verified”.
3. Przy kolejnym połączeniu stan = **VERIFIED**.
4. Symulacja ataku: usuń klucz serwera, wygeneruj nowy profil serwera — klient dostaje **MISMATCH** i blokadę.

11.3 Scenariusz 3: Transfer pliku

1. Po nawiązaniu sesji: przycisk **Send File**, wybór pliku.

2. Odbiorca w tym samym pokoju otrzymuje plik w katalogu pobrań.
3. Postęp: X/Y chunków w UI.
4. Przy symulowanym dropie chunka: `FILE_NACK` i retransmisja (testowane w `test_filetransfer.cpp`).

11.4 Scenariusz 4: Rotacja PFS

1. W trakcie sesji: Rotate Keys (ręcznie) lub poczekaj na auto-rotację serwera.
2. Nowy handshake na tym samym TCP; liczniki seq zerowane.
3. Stare wiadomości nie deszyfrują się nowym kluczem.

11.5 Scenariusz 5: Benchmark poziomów

1. Otwórz panel Benchmarks; uruchom dla FAST, BALANCED, MAX.
2. Porównaj `total_hs_ms` — FAST najszybszy, MAX najwolniejszy.
3. Oceń kompromis bezpieczeństwo vs. wydajność dla wybranego poziomu.

12 Wyniki wydajnościowe (orientacyjne)

Poniższe wartości pochodzą z wbudowanych benchmarków na typowym laptopie deweloperskim (Linux, Release build). Służą porównaniu poziomów — nie są gwarancją SLA.

Poziom	Keygen (ms)	Encaps (ms)	Sign (ms)	Handshake Σ (ms)
FAST	~ 0.5	~ 0.1	~ 0.5	$\sim 2\text{--}5$
BALANCED	~ 1.5	~ 0.3	~ 1.5	$\sim 5\text{--}12$
MAX	~ 3.0	~ 0.5	~ 2.5	$\sim 10\text{--}25$

Table 5: Szacunkowe czasy — zależne od CPU i liboqs. Mierzone w panelu Benchmarks.

AES-256-GCM na wiadomość tekstową ($<1\text{KB}$) to zwykle $<0.1\text{ms}$ — dominują koszty PQC przy handshake, nie szyfrowanie sesji. Dla komunikacji ciągłej po nawiązaniu tunelu opóźnienie jest akceptowalne nawet na poziomie MAX.

Wnioski praktyczne:

- **BALANCED** — sensowny domyślny wybór (Kyber-768 = rekomendacja NIST),
- **FAST** — gdy liczy się opóźnienie (np. demo na słabszym sprzęcie),
- **MAX** — gdy priorytetem jest maksymalna siła, kosztem czasu handshake.

13 Testowanie

13.1 Strategia testów

Testy w katalogu `tests/` linkują wyłącznie `quill_core` + Google Test. Brak testów GUI (headless CI). Pokrycie obejmuje:

- wektory referencyjne (RFC 5869 HKDF),
- testy negatywne (tampering, złe klucze, replay),
- testy integracyjne (handshake przez `socketpair`),
- testy wszystkich trzech poziomów bezpieczeństwa (parametryzacja GTest).

13.2 Katalog testów (105 przypadków)

13.2.1 `test_hkdf.cpp` (6)

`Rfc5869TestCase1`, `Rfc5869TestCase3`, `ClientServerSymmetry`, `DomainSeparationByLevel`, `TranscriptBinding`, `RejectsEmptyIkm`.

13.2.2 `test_argon2.cpp` (6)

`Deterministic`, `DifferentPassphraseDifferentKey`, `DifferentSaltDifferentKey`, `OutputLength`, `RandomSaltsDiffer`, `RejectsEmptySalt`.

13.2.3 `test_aesgcm.cpp` (13)

Roundtrip string/bytes, tamper ciphertext/tag, wrong key, unique nonce, bad key size, AAD roundtrip/wrong/missing/unexpected, AAD bytes, `ReplayBoundToSeqViaAad`.

13.2.4 test_kyber.cpp (10)

Parametryzacja FAST/BALANCED/MAX: EncapsDecapsSharedSecretMatches, FreshKeypairsPerCall, WrongSecretKeyGivesDifferentSecret; plus UnknownLevelThrows.

13.2.5 test_dilithium.cpp (13)

Parametryzacja $\times 4$ testy $\times 3$ poziomy + LevelAlgorithmMapping.

13.2.6 test_identity.cpp (10)

Plaintext persist, reload, encrypted save/load, wrong passphrase, encrypted without passphrase, QID1→QID2 migration, loose permissions, corrupted key, per-algorithm identities, fingerprint format.

13.2.7 test_profile.cpp (11)

Create/unlock, wrong/empty/short/repeated passphrase, duplicate name, path traversal, list profiles, failed create cleanup, unknown profile, remove requires correct passphrase.

13.2.8 test_roomstore.cpp (6)

Default general, protected room verify, persistence across reload, duplicate add throws, remove room, cannot remove general.

13.2.9 test_truststore.cpp (8)

First contact UNVERIFIED, second KNOWN, mark VERIFIED, algo rotation when allowed, MISMATCH preserves entry, remove allows fresh TOFU, distinct peers per host:port, corrupted known_hosts hard error.

13.2.10 test_messageformat.cpp (3)

Serialize roundtrip, binary payload all byte values, malformed throws.

13.2.11 test_filetransfer.cpp (14)

Multi-chunk roundtrip, corrupted chunk, missing chunk NACK, selective repeat recovery, max NACK exceeded, wrong final hash, duplicate chunks, cross-transfer injection, wrong chunk index, chunk_hash valid/wrong/missing, make_NACK indices, SHA3 known vector.

13.2.12 test_cryptomanager.cpp (5)

Same session key both sides, all security levels, re-handshake fresh key + KNOWN trust, server key change blocks, tampered KEM ciphertext rejected.

13.3 Uruchamianie testów

```
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -  
      DQUILL_BUILD_GUI=OFF  
cmake --build build -j$(nproc)  
./build/quill_tests           # bezpośrednio  
ctest --test-dir build --output-on-failure
```

13.4 CI GitHub Actions

Plik `.github/workflows/ci.yml`:

1. checkout z submodułami,
2. instalacja zależności Ubuntu (`libssl-dev`, `libargon2-dev`, `libgtest-dev`, `nlohmann-json3-dev`),
3. budowa i instalacja liboqs ze źródeł,
4. `cmake -DQUILL_BUILD_GUI=OFF`,
5. `ctest -output-on-failure`.

Każdy push/PR na `master/main` weryfikuje warstwę kryptograficzną.

14 Budowa i wdrożenie

14.1 Wymagania systemowe

Kompilator: C++23 (GCC 13+, Clang 16+).

Biblioteki:

- CMake ≥ 3.20 ,
- OpenSSL (dowolna wspierana; < 3.2 wymaga libargon2),
- liboqs (instalacja ze źródeł),
- nlohmann_json,
- Google Test (opcjonalnie, dla testów),
- GLFW3 + OpenGL (dla GUI).

14.2 Instalacja liboqs (Arch Linux)

```
git clone https://github.com/open-quantum-safe/liboqs
cd liboqs && mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(nproc)
sudo cmake --install .
```

14.3 Pełna budowa z GUI

```
git clone --recurse-submodules https://github.com/SmitchellD/Quill
.git
cd Quill
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(nproc)
./quill_tests      # 105/105
./Quill            # GUI
```

Jeśli brak submodułu ImGui:

```
git submodule update --init --recursive
```

14.4 Pakietowanie (opcjonalne)

Projekt nie dostarcza oficjalnego pakietu .deb/.pkg. Możliwe kierunki: Docker z `QUILL_BUILD_GUI=OFF` (tylko testy), AppImage z zależnościami GLFW — poza zakresem v1.0.0.

15 Analiza bezpieczeństwa — podsumowanie

15.1 Mocne strony

- Zero klasycznej asymetrii w protokole — zgodność z celem PQC.
- Fail-closed: TOFU MISMATCH, replay, GCM tag failure — wszystko odrzuca, nie degradowuje.
- Separacja `quill_core` — logika krypto testowalna bez UI.
- 105 testów z scenariuszami ataku (MITM, tamper, injection).
- Klucze at rest chronione Argon2id (memory-hard).
- PFS przez efemeryczny Kyber per sesję/rotację.

15.2 Słabe strony (akceptowane)

- Hub widzi plaintext — nie E2E.
- Brak PKI — TOFU wymaga dyscypliny użytkownika.
- Passphrase jedyna obrona offline — brak hardware token.
- Pliki w RAM przy wysyłce — limit rozmiaru.
- Brak forward secrecy dla tożsamości DSA (trwały klucz) — standardowe dla podpisów.

15.3 Porównanie z TLS 1.3 + hybryda PQC

Quill nie implementuje TLS. Różnice:

- TLS 1.3: klasyczny ECDHE domyślnie; hybrydy PQC dopiero się pojawiają.

- Quill: czysty ML-KEM + ML-DSA od początku; pełna kontrola nad transkryptem HKDF.
- TLS: PKI / certyfikaty; Quill: TOFU.
- TLS: dojrzały ekosystem; Quill: projekt akademicki, własny protokół.

16 Znane ograniczenia i dług techniczny

Obszar	Opis	Wpływ
E2E vs hub	Serwer odszyfrowuje i re-szyfruje	Brak poufności wobec hosta
NAT	Tylko TCP bezpośredni	Brak połączeń przez typowy router domowy bez przekierowania portu
PKI	Brak CA	Zaufanie TOFU + out-of-band fingerprint
RAM plików	Cały plik w pamięci nadawcy	Limit $\sim$ rozmiar RAM
JSON protokół	Brak wersji pola <code>version</code>	Trudniejsza ewolucja bez łamania kompatybilności
Wątki UI	Niektóre pola czytane bez mutex	Kosmetyczne race w licznikach UI
CI bez GUI	Brak testów renderowania ImGui	Regresje UI tylko manualnie
OpenSSL wersje	Dual backend Argon2	Złożoność build, ale pokryta CI

Table 6: Rejestr ograniczeń v1.0.0

17 Podsumowanie

17.1 Podział prac

Projekt zrealizowano w zespole dwuosobowym:

- **Szymon Kowalski** — kryptografia i protokół (`CryptoManager`, TOFU, HKDF), profile i `RoomStore`, testy automatyczne,

- **Michał Król** — warstwa sieciowa, interfejs ImGui (ChatApp/ChatAppRender), transfer plików, integracja UI z protokołem,
- **Wspólnie** — review bezpieczeństwa, dokumentacja techniczna, scenariusze testowe.

17.2 Osiągnięcia

Quill v1.0.0 dostarcza działający komunikator post-kwantowy z:

- pełnym handshake ML-KEM + ML-DSA/Falcon,
- trzema poziomami bezpieczeństwa (ustawianymi przez serwer),
- szyfrowaną sesją AES-256-GCM z anty-replay,
- TOFU fail-closed,
- profilami, pokojami z hasłem i szyfrowaniem kluczy Argon2id,
- transferem plików z SHA-3 i selective repeat,
- interfejsem ImGui z narzędziami edukacyjnymi,
- 105 testami i CI.

17.3 Możliwe kierunki rozwoju (poza zakresem studiów)

- NAT traversal (UDP hole punching + rendezvous),
- streaming plików bez pełnego buforu RAM,
- wersjonowanie protokołu JSON,
- prawdziwy end-to-end bez zaufania do serwera-relay.

A Załącznik A: Zmienne środowiskowe

Zmienna	Domyślnie
QUILL_PROFILES_DIR	~/.quill/profiles
QUILL_IDENTITY_DIR	~/.quill/identity

B Załącznik B: Rozmiary typowych artefaktów kryptograficznych

Poziom	Obiekt	Ok. rozmiar (B)
BALANCED	Kyber-768 public key	~1184
BALANCED	Kyber-768 ciphertext	~1088
BALANCED	ML-DSA-65 signature	~3309
BALANCED	ML-DSA-65 public key	~1952
Sesja	AES-256 key	32
Sesja	GCM nonce	12
Sesja	GCM tag	16

Table 7: Wartości orientacyjne — dokładne rozmiary zależą od liboqs.

C Załącznik C: Konwencje nazewnictwa peer_id w TOFU

Kierunek	Format peer_id
Klient identyfikuje serwer	srv:localhost:9999
Serwer identyfikuje klienta	cli:<SHA3-fingerprint>

D Załącznik D: Przykład zaszyfrowanej wiadomości CHAT (logiczny)

Po szyfrowaniu wiadomość na linii ma postać JSON (wartości przykładowe):

```
{
  "type": "CHAT",
  "sender": "Alice",
  "seq": 42,
  "timestamp": 1718123456789,
  "nonce": [/* 12 losowych bajtow */],
  "payload": [/* ciphertext + tag GCM, binarnie jako tablica */]
}
```

Przy deszyfracji odbiorca:

1. buduje AAD = "CHAT|42",

2. wywołuje `AesGcm::decrypt(key, nonce, payload, aad)`,
3. sprawdza `seq > recv_seq`, aktualizuje `recv_seq`.

E Załącznik E: Format pliku `rooms.json`

```
{
  "rooms": [
    { "name": "general", "protected": false },
    {
      "name": "secret",
      "protected": true,
      "salt": [/* 16 bajtow */],
      "verifier": [/* hash Argon2id */]
    }
  ]
}
```

Plik zapisywany atomowo w katalogu aktywnego profilu (`RoomStore`), `chmod 0600`.

F Załącznik F: Format pliku `known_hosts`

```
{
  "peers": [
    {
      "peer_id": "srv:localhost:9999",
      "fingerprint": "a3f29c1b44de...",
      "algo": "ML-DSA-65",
      "verified": true
    }
  ]
}
```

Plik zapisywany atomowo, `chmod 0600`. Uszkodzony JSON powoduje twardy błąd — brak cichego nadpisania (`test CorruptedKnownHostsIsHardError`).

G Załącznik G: Lista plików źródłowych `quill_core`

```
src/crypto/KyberKEM.cpp
src/crypto/DilithiumSign.cpp
```

```
src/crypto/AesGcm.cpp
src/crypto/Hkdf.cpp
src/crypto/Argon2.cpp
src/crypto/IdentityManager.cpp
src/crypto/ProfileManager.cpp
src/crypto/RoomStore.cpp
src/crypto/TrustStore.cpp
src/crypto/CryptoManager.cpp
src/network/Socket.cpp
src/network/NetworkServer.cpp
src/network/NetworkClient.cpp
src/protocol/MessageFormat.cpp
src/protocol/FileTransfer.cpp
```

References

- [1] NIST FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard. <https://csrc.nist.gov/pubs/fips/203/final>
- [2] NIST FIPS 204 — Module-Lattice-Based Digital Signature Standard. <https://csrc.nist.gov/pubs/fips/204/final>
- [3] NIST FIPS 205 — Stateless Hash-Based Digital Signature Standard (SPHINCS+). <https://csrc.nist.gov/pubs/fips/205/final>
- [4] Open Quantum Safe Project — liboqs documentation. <https://openquantumsafe.org>
- [5] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. Schanck, P. Schwabe, G. Seiler, D. Stehlé. *CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM*.
- [6] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, D. Stehlé. *CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme*.
- [7] H. Krawczyk. RFC 5869 — HMAC-based Extract-and-Expand Key Derivation Function (HKDF). 2010.
- [8] A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson. RFC 9106 — Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work. 2021.
- [9] T. Ylonen, C. Lonvick. RFC 4251 — The Secure Shell (SSH) Protocol Architecture. 2006.

- [10] NIST Post-Quantum Cryptography Standardization. <https://csrc.nist.gov/projects/post-quantum-cryptography>